

Remote Objects over Request-Driven Links with a Very Small MTU: A Layered Middleware Architecture

Florian Kaiser
Hochschule Karlsruhe
kaf11018@h-ka.de
Student ID: 79728

Abstract— Many embedded peripherals are attached over links that share three properties: only one side may start a transfer, the transferable block is small and of fixed size, and at least one of the two runtimes offers neither a network stack nor threads. Bluetooth Low Energy attribute access, I²C peripherals, Modbus over serial lines and several fieldbuses all fall into this class. Established remote invocation frameworks assume a transport that carries messages of arbitrary length and a back channel on which the callee may send at will, so none of them applies directly. This paper presents a middleware architecture that offers remote objects across such a link. The design is organized in five layers, keeps everything link-specific below a single-operation interception point, and derives the framing fields from the properties they have to guarantee rather than from a fixed wire format. Three design decisions carry the architecture: an explicit payload-length field, which is what allows the serialization format to be exchanged at all; a client-driven back channel that serves results and events through the same mechanism; and generational handles for remote objects, which make a stale reference detectable instead of silently addressing a different object. A cost model relates block size, header width and message length to the number of transfers, which on such links dominates the duration of a call. An instantiation on a 16-byte, request-driven link confirms the model: the duration of a call is the number of round trips times a constant, and reducing header width and switching to a binary encoding together cut the transfers per call by a factor of 2.09.

Index Terms—middleware, remote procedure call, embedded systems, constrained devices, framing, serialization, master-slave communication

I. Introduction

Attaching a peripheral to an embedded host is usually a question of drivers, not of architecture. That changes as soon as the peripheral carries state of its own, such as a graphical user interface, a configurable sensor front end or a motor controller with its own control loop. The host then no longer reads a value, it operates objects that live on the other side of a link, and the natural programming model for that is remote invocation [1].

The links used for such peripherals, however, rarely satisfy what remote invocation frameworks expect. Three properties recur:

1. **Request-driven transfer.** The link is usable in both directions, but only the superordinate side may start a transfer; the subordinate side answers and never speaks first. This holds for I²C peripherals [2], for Modbus over a serial line [3] and, in its common usage, for attribute reads in Bluetooth Low Energy [4].
2. **A very small, fixed block size.** One transfer carries a handful of bytes. The default attribute protocol MTU in Bluetooth Low Energy is 23 bytes [4], of which 20 remain for a value. A message of ordinary size therefore never fits into a single block, and the control information attached to each block becomes a leading cost factor rather than a rounding error.

3. **A constrained runtime.** At least one side runs on a constrained node in the sense of RFC 7228 [5]: no complete network stack, no threads in the usual sense, and little memory.

Individually, each property is easy to handle. Small blocks can be bridged by fragmentation, a missing send capability by polling, and a sequential program can be written without concurrency. Taken together they reinforce one another. Fragmentation multiplies the number of blocks, the request-driven link turns every one of those blocks into a separate request with its own answer, and the absence of concurrency serializes all of them. The result is that the duration of a call is governed by the number of transfers and barely at all by the speed of the link, which is an unusual starting point for a middleware design.

Established frameworks do not cover this combination. ONC RPC [6] and CORBA [7] assume a network transport, gRPC [8] assumes an HTTP/2 stack and bidirectional streams, and CoAP [9] standardizes the message level while leaving framing over an arbitrary block-oriented link open. The layer that is missing is the one directly above the link.

Contribution. This paper describes an architecture for remote objects over links of the class above. Specifically, it contributes (i) a five-layer decomposition that confines every link-specific

decision below a single-operation interception point; (ii) a derivation of the required framing fields from the guarantees they have to provide, including the argument that an explicit payload-length field is what makes the serialization format exchangeable; (iii) a client-driven back channel that delivers results and events through one mechanism; (iv) an object layer whose references are generational handles, so that a reference to a deleted object is rejected rather than silently rebound; and (v) a cost model that predicts the number of transfers per call, together with an instantiation that confirms it.

Section II states the system model and the requirements, Section III positions the work, Section IV develops the architecture, Section V reports an instantiation, and Sections VI and VII discuss limitations and conclude.

II. Problem Statement

A. Link and Runtime Model

The architecture assumes a link with the following properties and nothing beyond them.

The link transfers a block of fixed size M (the MTU) from the superordinate to the subordinate side and returns a block of the same size in the same operation. Only the superordinate side may start such an operation. Blocks may be lost, and the link is assumed to reject corrupted blocks by a checksum of its own rather than to deliver them damaged. Blocks are not reordered, because the channel is sequential, but a repeated block may arrive twice.

Following the roles on the link, the superordinate side is called the **client** and the subordinate side the **server**. The client runs the application and starts every transfer; the server holds the objects and executes the calls. The roles are fixed and do not change during operation, not even for events, since those are polled by the client as well.

Of the two runtimes, at least one is constrained: integers may be limited in width, threads may be unavailable, parts of the standard library may be missing, and memory is scarce. Shared code must therefore be written for the narrower of the two environments.

B. Requirements

ID	Requirement
C1	Transfer messages of arbitrary length over blocks of fixed size M .
C2	Deliver a result for every call and make it attributable to that call.
C3	Return events that originate on the server without losing them silently.
C4	Keep the response to a user-visible event within a stated bound.
C5	Bound the memory required for buffers at design time.
C6	Keep everything link-specific in one exchangeable place.

Table 1: Requirements on the communication architecture.

Table 1 lists the requirements the architecture has to satisfy. C1 follows from the block size, C2 and C3 from the request-driven link, C4 from interactive use, C5 from the constrained runtime

and C6 from the goal of covering a class of links rather than a single bus.

C3 deserves a remark. Because the server cannot send on its own, an event that occurs between two queries has to be buffered, and by C5 that buffer is bounded. Loss can therefore not be ruled out; what the architecture can require is that loss be **detectable** by the application rather than silent.

III. Related Work

Remote invocation. The idea of invoking a procedure that executes elsewhere, while the call site looks local, goes back to Birrell and Nelson [1] and underlies most middleware since. Waldo et al. [10] argue that this transparency is necessarily incomplete: latency, partial failure and the absence of shared memory remain visible. That argument is sharper here than in a local network, because the latency gap is several orders of magnitude. Object-oriented middleware extends the idea from a fixed set of procedures to objects created at run time and addressed by reference [7]; the client-side representative follows the Proxy pattern [11].

Among concrete frameworks, ONC RPC [6] pairs a protocol with its own serialization format [12] and targets networks rather than channels of a few bytes. Thrift [13] and gRPC [8] are schema-bound and aimed at service infrastructure; gRPC in particular requires an HTTP/2 stack and a bidirectional connection. All of them assume a transport that carries messages of arbitrary length and a channel on which the server may send at will. Neither assumption holds here.

Serialization. Text-based formats such as JSON and XML are readable and self-describing but spend bytes on keys and structural characters. Binary formats such as CBOR [14] and MessagePack [15] encode the same data model more compactly while remaining schema-free; schema-bound formats such as Protocol Buffers [16] are smaller still, at the price of a shared interface description and a compilation step. One property of binary formats matters disproportionately on this link: their output may contain any byte, including the zero byte, so a receiver cannot infer the payload length by stripping padding. Section IV-C returns to this point.

Protocols for constrained nodes. RFC 7228 [5] provides the terminology for devices of this size. CoAP [9] is the closest relative in spirit, a compact request/response protocol for constrained networks, and its Observe extension [17] adds server-initiated notifications, which this link cannot support. CoAP, however, is defined over a datagram transport and standardizes the message level; the framing of a message across a link whose MTU is smaller than any realistic message remains outside its scope. The same holds for MQTT [18] and its sensor-network variant [19], which additionally presuppose a broker and a connection the client keeps open.

Gap. The capable frameworks assume a transport the target class does not provide; the lean protocols standardize the message level and leave framing, fragmentation and the back

channel open; and the domain-specific bus protocols solve a different problem or assume the subordinate side can send on its own. The layer between a block-oriented, request-driven link and an object model is the one this paper addresses.

IV. Architecture

A. Layering

The architecture is organized as a middleware stack with remote invocation. Table 2 shows the five layers, which exist on both sides and differ only in the direction from which a call arrives. Each layer talks exclusively to the one immediately below it, so a change inside a layer does not affect the others.

Client	Server
Application	Peripheral logic
Object layer: proxies	Object layer: adapters, registry
Communication layer	Communication layer
Transport layer	Transport layer
Link binding	Link binding
Request-driven link, fixed block size M	

Table 2: Layering of the architecture. Both sides are built alike; each layer communicates only with the layer directly below it.

The decomposition exists to confine the three difficult properties of the link. The block size is absorbed by the transport layer, the restricted direction of communication by the communication layer, and the absence of a link-independent formulation by the link binding. Above the object layer, none of the three is visible.

B. Link Binding and the Interception Point

The link binding is the only layer that knows the actual transfer path. Its task is to carry one block of fixed length to the other side and to accept one such block in return, without knowing the content of either.

Upward it therefore offers a single operation: a block is handed over and, in the same operation, a block comes back. Whether that block carries payload, an acknowledgement or a request for a result is irrelevant to the operation, which only conveys it. This narrowness is deliberate and is what satisfies C6. A richer interface would exclude links: delivery without a preceding request presupposes that the server may send on its own, and waiting for a server notification additionally presupposes that it may do so at a moment of its own choosing. Neither can be assumed for the class in Section II, so the interface is the intersection of what all links of that class can do. Binding a different link means providing one further implementation of this single operation.

C. Transport Layer

The transport layer serializes a message, divides it into blocks, attaches control information to each block and reassembles the message on the other side. Upward it hides the block size completely.

Framing fields. The architecture prescribes which information the header must carry and why, not how wide the fields are or in

which order they appear. Table 3 lists the fields and the property each one guarantees.

Field	Guarantee
Message ID	Decides whether an arriving block belongs to the current transfer. Needed even for a strictly synchronous exchange, because a read that immediately follows a write may still return the answer to the previous request.
Position	Orders the blocks for reassembly and identifies a repetition.
End marker	Ends the message; the receiver does not know its total length in advance.
Payload length	States how many bytes of the block belong to the payload.
Frame kind	Separates payload from control traffic on the one channel available.

Table 3: Framing fields and the guarantee each provides.

Payload length and format independence. The payload-length field is the one field whose necessity is easy to underestimate, and it decides a property of the whole architecture. The link delivers blocks of fixed size, but the payload in a block is not always that long. A missing length is harmless as long as the end of the payload follows from its format, which is the case for text-based formats: they contain no zero bytes, so padding at the end of a block can be recognized and removed. Binary formats such as CBOR [14] or MessagePack [15] do not have this property. Without an explicit length the transport layer is therefore bound to text-based formats, and the choice of encoding is no longer free. Declaring the length of the payload is not unusual in itself; HTTP, for instance, lets a message state the media type of its payload for the same reason [20].

The width of the field follows from the block size. With M the block size and H the header size, both in bytes, the payload per block is

$$P = M - H \quad (1)$$

and the length field must be able to express every value from 0 to P , so its width b in bits is

$$b = \lceil \log_2(P + 1) \rceil. \quad (2)$$

Equations (1) and (2) depend on each other, since H determines P , P determines b , and b is part of H again. The dependency is resolved by choosing the smallest header for which both equations hold simultaneously.

Cost model. On a link of this class the relevant unit is not the byte but the block. A message of length L decomposes into

$$N = \left\lceil \frac{L}{P} \right\rceil \quad (3)$$

blocks, and $N \cdot M$ bytes are transferred regardless of how much of that is used. The payload efficiency

$$E = \frac{L}{N \cdot M} \quad (4)$$

therefore depends on three factors: block size, header width and payload length. Header width and payload length both act through the same rounding, but differently. A shorter payload lowers N only when it crosses a block boundary, whereas a smaller header moves every block boundary at once. For a small M one should thus expect framing to affect the number of transfers more strongly than the choice of serialization format, even though the format reduces the payload more directly. Section V tests this expectation.

Reassembly and repetition. A block is retransmitted when its acknowledgement fails to appear, which may mean that the block was lost, that the link discarded it, or that the acknowledgement itself was lost. The sender cannot distinguish these cases and reacts identically in all of them. Since the decomposition of a message is deterministic, a repeated block carries the same content as the original, but that alone does not help the receiver: appending it a second time would produce a wrong message. Reassembly must therefore discard blocks whose position is already present, which is what makes it idempotent.

Bounded buffers. The receiver has to hold the blocks of a message until it is complete, and C5 requires a bound known at design time. It follows from the fields already fixed: if b_{pos} is the width of the position field, $2^{b_{\text{pos}}}$ positions can be distinguished and the longest transferable message is at most

$$L_{\text{max}} = P \cdot 2^{b_{\text{pos}}} \quad (5)$$

bytes, which is at the same time the upper bound of the buffer the other side must provide.

Non-goals. Three tasks are deliberately left to the link: error correction, which is replaced by retransmission on top of the link's own checksum; ordering, because the channel is sequential and the position field makes reassembly order-independent anyway; and congestion avoidance, which cannot arise on a point-to-point link with one master and one message in flight.

D. Communication Layer

The communication layer establishes the remote call. Of the transport layer it knows only two operations, sending a message and receiving one; how many blocks the message occupies, how often a block is repeated and in which format the payload is encoded remain hidden.

What a call carries. A call must carry enough for the server to execute it without a further question, which requires three items: a method identifier for what is executed, the arguments for the values it is executed with, and a **scope** that selects which receiver executes it. Two decisions keep this short, which matters because by (3) the length directly determines the number of blocks: arguments are transferred positionally rather than by name, and the method is denoted by an identifier of fixed width that both sides derive from its name in the same way, so no table has to be maintained.

The result takes the same way back. It is a message as well and carries either the return value or an error, distinguishable by the key under which the content appears.

Request-driven back channel. Because the server cannot send, the result is not delivered but held and handed out on request. The client first asks whether a result is ready and then fetches it block by block, each request naming the next position, so that a repeated request yields the same block. The communication layer hides this, so that upward a call appears as an ordinary synchronous invocation that returns once the result is available.

Events use the same mechanism rather than one of their own. They are buffered per object on the server and read by an ordinary call. The temporal resolution is thus bounded by the polling interval: an event occurring between two queries is noticed at the next one. Polling more often shortens that delay but costs one transfer per query, and polling less often saves transfers and lengthens the delay, so the two goals cannot be improved at once. If the bounded buffer overflows, the oldest event is discarded and counted, and the count is returned with the next query. Loss thereby becomes visible to the application, as C3 demands.

Correlation and repetition limits. Every call carries the message identifier of its request, which is what lets the client recognize an answer to a previous request as stale. Since the client sends strictly one message at a time, the identifier may repeat after a fixed number of calls, as long as no older result can still be in flight between two occurrences. Retransmissions need an upper bound: without one the client waits indefinitely when the server stops answering, turning a fault on one side into a standstill on the other. With a bound, the call fails after a known time, which is also the precondition for C4 to be checkable at all, because a response time can only be stated if it is bounded above.

E. Object Layer

The object layer maps the entities of the peripheral onto objects the application uses like local ones. Each object type has a common interface that is realized on the client by a **proxy**, which forwards every method call, and on the server by an **adapter**, which executes it. That both sides satisfy the same interface is the basis of call transparency: the application cannot tell a proxy from a local implementation [11].

A **dispatcher** assigns an incoming message to the method to be executed. The scope selects the responsible dispatcher, and the method identifier selects the method within it. One dispatcher can be registered per object type, which decouples the routing of messages from the implementation of methods and allows the object model to grow without touching the layers below.

References as generational handles. Objects are created at run time, so the server issues a reference for each one and the client passes it along with every call. A reference that merely names a slot in a table is unsafe: after the object is deleted, the slot may be reused, and a proxy that still holds the old reference would silently operate on the new object. The architecture therefore uses a generational handle, a single integer composed of a slot and a generation counting how often that slot has been occupied. Releasing an object advances the generation, so a handle naming

the previous generation is recognized as stale even though its slot is valid again. The construction is the distributed-object counterpart of the tombstone and lock-and-key schemes used to detect dangling pointers in local memory [21], [22]. A single integer is used rather than a pair because it costs the fewest bytes on the wire, which by (3) is what matters.

Two further properties make the scheme usable in practice. First, each entry records the **kind** of its object, and every method states the kind it expects, so passing a reference of the wrong type is rejected before the object is touched. Second, objects are created inside a parent object, and the registry mirrors that tree, so that releasing a parent releases everything created inside it. Without the tree, child entries would survive their objects.

The client additionally maintains a **session** that ends when the object graph is cleared and when the link fails. Each proxy remembers the session it was created in and raises the error locally, without starting a transfer, if it belongs to an earlier one. This is the only reliable check after a link failure, because the server may have restarted in the meantime and rebuilt its registry from scratch. So that an old reference does not match by accident in that case, generations start at a random value after a restart.

Distinguishable errors. Both checks report the same way: the error payload carries a description and a kind, and the client raises the exception corresponding to that kind, distinguishing at least a stale reference, a reference of the wrong kind, and any other remote error. The application can then react to a stale reference specifically, for instance by rebuilding the interface, without parsing an error text.

F. Concurrency, Errors and Local Bindings

On the client there need be no concurrency at all: a call blocks the application until the result arrives or the retransmission bound is reached. Since only one call is in flight, no locks and no buffers for several open calls are required, which suits a runtime without threads.

On the server, three activities coexist: answering the client's requests, running the peripheral's own periodic work, and executing received calls. They share one cooperative event loop rather than separate threads. This is not only a concession to constrained runtimes; peripheral libraries are frequently non-reentrant, and a preemptive thread that enters such a library while its own periodic work is running is a defect that is hard to reproduce.

Errors are handled in the layer in which they occur and reach the layer above only if they cannot be handled locally. Two rules bound the behaviour. First, every call leaves either a result or an error in the output buffer, so the client, which waits for exactly one answer, also receives one in the failure case. Second, a callback the peripheral library invokes must never propagate an exception, because an unhandled exception there ends the event loop permanently while calls keep arriving; a failed read is reported as "no input" instead.

Finally, the architecture admits **local bindings**: an event of a server-side object may be tied to an action on another server-

side object, executed on the server without involving the client. A binding costs no transfer at all, which is the only way to satisfy a tight bound in C4, at the price of a fixed repertoire of actions. The mechanism is an explicit trade between latency and generality, and it is possible precisely because the object layer is symmetric: the action is the same method the client would otherwise call.

V. Instantiation

To check that the architecture is realizable and that the cost model holds, it was instantiated on a link of the target class: a request-driven serial connection with a block size of $M = 16$ bytes between a control unit running a reduced Python runtime without threads or long integers, and a peripheral unit running a microcontroller Python port with a graphical library. The object model exposed screens, labels and buttons. All measurements below were taken with logging disabled, since synchronous console output would otherwise dominate the timings.

Sizing. With $M = 16$, a three-byte header is the smallest one that satisfies (1) and (2) simultaneously: it leaves $P = 13$ bytes of payload, for which (2) requires a four-bit length field, and that field shares a byte with the frame kind and so costs nothing extra. A two-byte header would have been possible only by narrowing the message identifier or the position field, reducing either the number of distinguishable messages or, by (5), the maximum message length. With an eight-bit position field and one value reserved for control frames, (5) yields 3315 bytes as the buffer bound required by C5. Handles were 16 bits wide, ten for the slot and six for the generation, which admits 1024 simultaneous objects and costs at most three bytes when encoded.

Cost per call. Across 640 calls with payloads from 21 to 142 bytes and no retransmissions, one round trip took 114.1 ms with a standard deviation of 0.3 ms, independent of payload length and encoding. The duration of a call is therefore the number of round trips times a constant: the shortest observed call needed 4 round trips and 457 ms, the longest 27 round trips and 3082 ms. Two of those round trips are spent on fetching the result and do not depend on the message length, which for short calls is about half of the total time. Payload efficiency by (4) stayed between 0.66 and 0.81, but it is the number of blocks, not their filling, that governs the duration.

Framing versus encoding. The instantiation was measured against an earlier variant of itself with a larger header ($P = 7$) and against a text-based encoding, in a two-by-two design. Table 4 shows the result. Both factors act in the predicted direction, and framing outweighs the encoding, which is what Section IV-C leads one to expect: the encoding shortens the payload, but only the header moves every block boundary at once.

Variant	Round trips, relative
Binary encoding, $P = 13$ (reference)	1.00
Binary encoding, $P = 7$	1.56
Text encoding, $P = 13$	1.18
Text encoding, $P = 7$	2.09

Table 4: Transfers per call relative to the reference variant, same calls and same payloads in all four runs.

Reaction time. An input on the peripheral that is answered by a local binding was handled in at most 51 ms end to end. The same reaction routed through the client took about 1490 ms, since it costs two full calls, one to poll the event and one to act on it. This is the sharpest consequence of the architecture: interaction that must be fast has to be expressible in the binding repertoire, and everything else pays two calls.

Footprint. On the constrained client, an interface of two screens, two labels and two buttons occupied 144 bytes beyond the program itself, since a proxy holds only a reference, a kind and a session. The registry, the event buffers and the reassembly buffer all live on the server, where the bound from (5) applies.

VI. Discussion and Limitations

Transparency is syntactic, not temporal. A call looks local but takes three orders of magnitude longer, which is precisely the limitation Waldo et al. [10] describe. The architecture does not remove it; the local-binding mechanism is a way of routing around it for the cases where it matters most.

The back channel bounds interactivity. Because the server cannot send, the shortest possible reaction through the client is two calls. Where that is too slow and no binding fits, the architecture offers nothing better. An obvious refinement is to return the result of a short call together with the acknowledgement of its last block, which would remove the two fetch round trips; another is to aggregate the events of all objects into a single query, so that the number of queries stops growing with the number of interactive elements.

Blocking clients. With no concurrency on the client, the application halts for the duration of a call. Where the runtime offers cooperative multitasking, only the proxy interface and the waiting loop would have to change; the request-driven back channel is unaffected, since the client still starts every transfer.

Generations are finite. A generation of g bits repeats after 2^g releases of the same slot, after which a long-lived stale proxy matches again. Widening the field costs bytes on the wire in every call. Moreover, the client only knows the proxies it holds itself, so deleting a parent leaves its children’s proxies locally valid until their next call is rejected by the server.

Schema-free encoding defers checking. Whether a method exists and whether its arguments fit is decided at run time. If the two sides use different encodings, the payload still decodes but no longer yields a valid call; the architecture can report that clearly but cannot prevent it.

Scope of the evidence. The cost model and the reaction times were obtained on one instantiation of one link. The model itself contains nothing link-specific, and the four-variant comparison varies exactly the two quantities it predicts, but a second instantiation on a different link of the same class would strengthen the claim considerably.

VII. Conclusion

Links that are request-driven, carry only a few bytes per transfer and terminate in a constrained runtime are common in embedded systems, yet they fall outside what established remote invocation frameworks assume. This paper presented a middleware architecture that nevertheless offers remote objects across such a link. It confines every link-specific decision below a single-operation interception point, derives the framing fields from the guarantees they must provide, serves results and events through one client-driven back channel, and makes references to remote objects generational so that a stale reference is rejected rather than silently rebound.

The governing quantity on such links is the number of transfers per call, not the speed of the link. The cost model relates that number to block size, header width and message length, and an instantiation on a 16-byte link confirms it: a round trip is constant, the duration of a call is a multiple of it, and header width and encoding together change the number of transfers by a factor of 2.09, with the header contributing the larger share. The architecture’s own limits follow from the same model. A reaction routed through the client costs two calls, which is why an action that must be fast is executed on the server as a local binding.

Future work follows directly from these limits: returning the result of a short call with the acknowledgement of its last block, aggregating events of all objects into a single query, and instantiating the architecture on a second link of the class, for which attribute access in Bluetooth Low Energy is the natural candidate.

AI Usage Disclosure. AI tools (Claude, Anthropic) were used to assist in drafting and editing the text of this paper. The architecture, its instantiation, all measurements and the final text were produced and verified by the author.

References

- [1] A. D. Birrell and B. J. Nelson, “Implementing Remote Procedure Calls,” *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, 1984, doi: 10.1145/2080.357392.
- [2] NXP Semiconductors, “I2C-bus Specification and User Manual, UM10204, Rev. 7.0,” technical report, 2021.
- [3] Modbus Organization, “MODBUS over Serial Line Specification and Implementation Guide V1.02,” technical report, 2006.
- [4] Bluetooth SIG, “Bluetooth Core Specification, Version 5.4,” technical report, 2023.

- [5] C. Bormann, M. Ersue, and A. Keränen, “Terminology for Constrained-Node Networks,” no. RFC7228. 2014. doi: 10.17487/RFC7228.
- [6] R. Thurlow, “RPC: Remote Procedure Call Protocol Specification Version 2,” no. RFC5531. 2009. doi: 10.17487/RFC5531.
- [7] Object Management Group, “Common Object Request Broker Architecture (CORBA) Specification, Version 3.4,” technical report, 2021.
- [8] The gRPC Authors, “gRPC: A High Performance, Open Source Universal RPC Framework.” 2024.
- [9] Z. Shelby, K. Hartke, and C. Bormann, “The Constrained Application Protocol (CoAP),” no. RFC7252. 2014. doi: 10.17487/RFC7252.
- [10] J. Waldo, G. Wyant, A. Wollrath, and S. Kendall, “A Note on Distributed Computing,” technical report TR-94–29, 1994.
- [11] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.
- [12] M. Eisler, “XDR: External Data Representation Standard,” no. RFC4506. 2006. doi: 10.17487/RFC4506.
- [13] M. Slee, A. Agarwal, and M. Kwiatkowski, “Thrift: Scalable Cross-Language Services Implementation,” technical report, 2007.
- [14] C. Bormann and P. Hoffman, “Concise Binary Object Representation (CBOR),” no. RFC8949. 2020. doi: 10.17487/RFC8949.
- [15] S. Furuhashi, “MessagePack: It's Like JSON, but Fast and Small.” 2024.
- [16] Google LLC, “Protocol Buffers Documentation.” 2024.
- [17] K. Hartke, “Observing Resources in the Constrained Application Protocol (CoAP),” no. RFC7641. 2015. doi: 10.17487/RFC7641.
- [18] OASIS, “MQTT Version 5.0, OASIS Standard,” technical report, 2019.
- [19] U. Hunkeler, H. L. Truong, and A. Stanford-Clark, “MQTT-S – A Publish/Subscribe Protocol for Wireless Sensor Networks,” in *3rd International Conference on Communication Systems Software and Middleware (COMSWARE)*, 2008, pp. 791–798. doi: 10.1109/COMSWA.2008.4554519.
- [20] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” no. RFC9110. 2022. doi: 10.17487/RFC9110.
- [21] D. B. Lomet, “Scheme for Invalidating References to Freed Storage,” *IBM Journal of Research and Development*, vol. 19, no. 1, pp. 26–35, 1975, doi: 10.1147/rd.191.0026.
- [22] C. N. Fischer and R. J. LeBlanc, “The Implementation of Run-Time Diagnostics in Pascal,” *IEEE Transactions on Software Engineering*, no. 4, pp. 313–319, 1980, doi: 10.1109/TSE.1980.234494.